

Permutation Substring Search

Topic.

String algorithms, sliding window, frequency counting.

Statement.

Given strings s and t . Find all substrings of t that are permutations of s .

Solution.

We move a window of length $|s|$ along the string t .

A substring $t[l, r)$ is suitable if and only if it is a permutation of the string s , that is, for every symbol the number of its occurrences in $t[l, r)$ is equal to the number of its occurrences in s .

Let

$$|s| = m, \quad |t| = n.$$

We store:

$$cnt_s[c]$$

the number of times symbol c occurs in s ,

$$cnt[c]$$

the number of times symbol c occurs in the current window of length m ,

$$bad$$

the number of symbols for which

$$cnt[c] \neq cnt_s[c].$$

Then the current window is suitable if and only if

$$bad = 0.$$

We do preprocessing.

First, compute cnt_s for the string s .

Then compute cnt for the first window

$$t[0, m).$$

After that, for all symbols that occurred in s or in the first window, check whether the frequencies are equal and compute bad .

Then we move the window one position to the right:

$$t[l, r) \rightarrow t[l + 1, r + 1).$$

Then:

$$x = t[l]$$

leaves the window, and

$$y = t[r]$$

enters the window.

Only the values $cnt[x]$ and $cnt[y]$ change, so only they can affect bad .

Update.

Before changing the frequency, check whether the symbol was bad:

$$was = [cnt[c] \neq cnt_s[c]].$$

Change the frequency.

After changing it, check whether the symbol has become bad:

$$now = [cnt[c] \neq cnt_s[c]].$$

Update bad :

$$bad+ = now - was.$$

If after the next shift

$$bad = 0,$$

then the current window is a permutation of the string s .

Otherwise, it is not.

For every window, we store exact frequencies of all symbols in this window. A window is a permutation if and only if all its symbol frequencies coincide with the frequencies in s , which is equivalent to

$$bad = 0.$$

When the window shifts, only two symbols change, so it is enough to update only them.

Complexity.

Building cnt_s takes

$$O(m).$$

Building the first window cnt takes

$$O(m).$$

Each shift takes

$$O(1).$$

There are n shifts in total, so the total running time is

$$O(n).$$

For memory, we store frequency tables for symbols that occurred in s and t .

In the worst case, the memory consumption is

$$O(n).$$